AI vs Human Ludo

INTERVIEW PREPARATION GUIDE

Complete answer scripts for the most common project questions

Full-Stack Development + AI | Simple explanations with real technical depth

PREPARED FOR A SOFTWARE ENGINEERING INTERN ROLE AT BNY

In this guide

- 26 core project questions (Q1 - Q26) - every question in your list, plus 'Tell me about yourself'
- 6 bonus technical questions interviewers commonly add (CORS, scaling, SVG, testing, and more)
- 5 HR and behavioural questions reported in real BNY internship interviews
- A 1-page architecture summary and a cheat sheet of every number you should quote
- Each script: why they ask, your 30-60 second answer, and follow-up probes

Based on a full review of your actual codebase + web research of BNY intern interviews

How to use this guide

- Do not memorize word-for-word. Learn the structure - direct answer, then why, then a project detail - and retell it in your own words. Read each script aloud 2-3 times.
- 'Why they ask' shows the intent. If you forget the details mid-answer, answer the intent - interviewers grade understanding, not recitation.
- 'If they probe' lines are only for follow-ups. Never volunteer them - let the interviewer ask.
- Quote only the cheat sheet numbers. It contains every figure you should say out loud. Never invent numbers in the interview.

Golden rules for the interview

- The three-beat answer: (1) the direct answer, (2) the reason, (3) a concrete detail from this project.
- Pause for two seconds before answering - it looks thoughtful, not slow.
- If you do not know: say 'I have not done that yet, but here is how I would approach it...' then give your thinking. This answers the intent while staying honest.
- Never claim things the code does not do. Your honest answers (Q12, Q13, Q18, B6) are designed to impress - interviewers respect awareness more than overclaiming.
- About AI tools, be transparent: 'I used AI tools to explore approaches and speed up boilerplate, but I directed the design, understood every line, and fixed the bugs it produced.' Then prove it by going deep - which this guide lets you do.
- BNY grills projects deeply. A reported 2025 interview: the interviewer 'asked about each and every detail' of the project, the tech stack, how to deploy it - and asked for code snippets. Know this guide back to front.
- End with a hook: 'Would you like me to go deeper into the scoring?' - interviews are conversations, not interrogations.

What the BNY intern process looks like (from real interview reports)

Typical rounds (3 - 4 rounds over 2 - 5 weeks)

- 1. Online Assessment (HackerRank): about 2.5 hours, 3 - 4 DSA problems (arrays, recursion, greedy, DP, graphs) plus MCQs. Difficulty is medium-to-hard, but candidates with even one full solve have advanced.
- 2. Technical round(s): live coding plus fundamentals - OOP and its 4 pillars, DBMS and SQL, time and space complexity, OS and networking. Some rounds instead do a deep project dive.
- 3. HR / behavioural: 'Why BNY', ethics and integrity, teamwork and disagreement stories. Sometimes a written behavioural scenario.

Reported question themes

- Why are you interested in the company?
- How have you dealt with ethics and integrity?
- How did you manage team projects and people's skills?
- What is OOP? Explain the four pillars with real-life applications.
- Explain the time and space complexity of your approach.
- What is the most complex project you have worked on, and what was your specific contribution?
- How would you scale this project if the user base grew 10x?
- Tell me about a time you handled a disagreement in a team.

What they value: integrity, ethics, teamwork, scale and safety trade-offs, and deep project knowledge. This guide prepares the project deep-dive and the behavioural stories. DSA practice is separate - solve a few LeetCode problems daily alongside this.

Contents

Part 3 - HR and behavioural questions

H1 Why BNY Mellon

H2 What are your strengths and weaknesses

H3 Where do you see yourself in five years

H4 Tell me about a time you handled a disagreement or a conflict of priorities

H5 How have you dealt with ethics and integrity

Part 4 - Cheat sheet (memorize these numbers)

Part 5 - One-page architecture summary

Part 1 - Core project questions (Q1 - Q26)

Q1 - Tell me about yourself

Approx. answer time: 60-75 sec

WHY THEY ASK

The most important answer in the interview - hiring decisions are largely made in the first minute. Use the Past to Present to Future structure and stay under 90 seconds.

YOUR SCRIPT

I am preparing for a career in software development, and my journey has been hands-on. I do not come from a traditional computer science background, so instead of only studying theory, I learned by building - and the project on my resume, AI vs Human Ludo, is the proof. It is a complete full-stack application where a human plays Ludo against AI opponents. The backend is Python with FastAPI, and it holds every game rule and the AI engine. The frontend is React with TypeScript, and it renders the board and drives the game. The AI part is what I enjoyed most: I implemented three engines - Expectiminimax, Monte Carlo Tree Search, and a hybrid - and the game shows live win probabilities and the reasoning behind every move.

Through this project I learned the full stack end to end - APIs, state management, AI algorithms - and because I am still early in my journey, I am hungry to learn. I am excited about BNY because of the scale and trust its technology handles, and I would love to grow as an engineer there.

IF THEY PROBE FURTHER

Probe: Why software?

> I love building things you can see and use. The moment my board rendered and the AI started playing, I was hooked - and every feature since has felt like solving a puzzle.

Probe: Why this project?

> Because one project could teach me the whole stack - frontend, backend, APIs, and AI - and it forced me to learn systems thinking, not just syntax.

Q2 - Project introduction

Approx. answer time: 40-50 sec

WHY THEY ASK

Your elevator pitch - it sets the tone for everything after.

YOUR SCRIPT

This project is called AI vs Human Ludo. It is a web-based Ludo game where a human plays against AI-controlled opponents. It is built as two separate pieces talking over HTTP: a backend in Python using FastAPI, which owns all the game rules and the AI brain, and a frontend in React, which renders the board and lets you interact.

The AI is not random - on every roll it evaluates all legal moves, scores each one, and picks the best. A nice touch is the analytics panel: it shows what the AI is thinking in real time - win probability, the best move, and the reasoning behind it. I implemented three AI engines - Expectiminimax, Monte Carlo Tree Search, and a hybrid - and the live game runs on Expectiminimax. It is a complete full-stack demo: game logic, AI algorithms, REST APIs, and a polished UI.

IF THEY PROBE FURTHER

Probe: Why Python for the AI?

Python is great for algorithm-heavy logic - writing tree search and heuristics is very natural - and FastAPI makes exposing it as an API quick.

Probe: What was your specific contribution?

I built the whole thing end to end, but the parts I am most responsible for are the AI engine, the API design, and keeping the frontend and backend in sync.

Q3 - How do you figure out the score of a move

Approx. answer time: 45-60 sec

WHY THEY ASK

The heart of your AI. They want proof you understand the evaluation, not that you copied code.

YOUR SCRIPT

Whenever a player rolls the dice, the backend finds which tokens can legally move. For each legal move it computes a score using a heuristic - a scoring formula that grades how good the move is. Think of it as a coach grading a play: points are added for things you want and subtracted for things you do not.

Points are added for progress toward home, for capturing an opponent's token, for forming blockades, for sitting on safe star tiles, and a big bonus when a token reaches home. Points are subtracted when a move leaves a token in danger - meaning an enemy token is one to six tiles behind it, inside capture range.

Then comes the future-thinking step: for each token, the AI imagines what the opponent could do on all six dice outcomes and blends that into the score, so the final score is an expected value, not just a snapshot. The highest-scoring move becomes the AI's choice - and the same score is shown in the UI, so you can see exactly why the AI made its move.

IF THEY PROBE FURTHER

Probe: What are the exact weights?

Capture bonus 80, risk penalty 35, progress 2 per tile, safety 25, blockade 40, home bonus 200. And there are three personalities - aggressive, defensive, balanced - that change these weights to create different play styles.

Probe: Why the 1 to 6 tile risk window?

Because that is exactly the range an opponent can capture you from with a single dice roll - anything further is not immediate danger.

Q4 - How do other AI games work, and how is it different from your AI game

Approx. answer time: 50-60 sec

WHY THEY ASK

Tests field awareness - can you place your AI inside the landscape of how game AIs work?

YOUR SCRIPT

Most game AIs fall into a few categories. Chess engines like Stockfish search incredibly deep - millions of positions - using minimax with alpha-beta pruning, which prunes away branches that cannot matter, plus opening books and endgame tables. AlphaGo went further and used deep neural networks trained on millions of games to evaluate positions, because Go's branching factor is too big for brute-force search. Simple games like Tic-Tac-Toe are solved - you can search the whole game tree and never lose.

My Ludo AI is different in one key way: Ludo has dice, which means randomness. You cannot just search for the best move, because you do not control the roll. So I used Expectiminimax, which adds chance nodes: it scores a

move, simulates what the opponent could do on all six dice outcomes, and averages the worst cases into an expected value. I also did not use machine learning - my heuristic is hand-crafted with weights for capture, safety, progress, and risk. And unlike a black box like AlphaGo, my AI is transparent - the UI shows the reasoning behind every move.

IF THEY PROBE FURTHER

Probe: What does depth mean here?

How many moves ahead we look. Chess engines look 20 to 40 plies. Mine looks one move ahead plus the opponent's reply, because Ludo's branching - 4 tokens times 6 dice - explodes quickly.

Probe: Could you have used a neural network?

Yes, people have trained AIs for Ludo-style games, but it needs data, compute, and time. For a small game, a transparent heuristic is simpler, instant, and explainable - that is the trade-off I accepted.

Q5 - Explain the score in math notation, start and end points, positive and negative reward

Approx. answer time: 50-60 sec

WHY THEY ASK

They want to confirm you understand the math behind the score, not just the code.

YOUR SCRIPT

First, the position system - think of it as a number line from start to goal. Each token sits at a number from -1 to 57. -1 is the start: the token is still in its base. 0 to 51 is the main track that loops around the board. 52 to 56 is the token's own home stretch, and 57 is the goal - home. So the distance to goal is simply 57 minus the position.

The score is a sum of positive and negative rewards. In simple math notation: Score = (position x 2) + 30 if in home stretch + 200 if home + 25 if on a safe tile + 40 per blockade + 80 per capture, minus 35 if an enemy is 1 to 6 tiles behind.

The positive rewards encourage progress, safety, captures, and finishing. The one negative reward is risk: an opponent within capture range behind you makes the token unsafe, so we subtract points. And the final score of a move is an expected value - the average across all six dice outcomes - so randomness is baked into the math instead of left to luck.

IF THEY PROBE FURTHER

Probe: Why those exact weights?

They are tuned to create behaviour: aggressive AI raises the capture weight, defensive AI raises risk and safety weights. I can flip personalities and see the play style change.

Probe: What is expected value?

The probability-weighted average of all outcomes - like the average of a dice roll is 3.5 even though it is never exactly 3.5.

Q6 - What is Vite and what is HMR

Approx. answer time: 40-50 sec

WHY THEY ASK

Frontend fundamentals - every full-stack intern is expected to know the tooling.

YOUR SCRIPT

Vite is a modern build tool for frontend projects - it is the engine that runs my React app during development and packages it for production. The clever part is how it works in development: instead of bundling the whole app whenever I change a file, it serves modules on demand, so the dev server starts almost instantly, even for big projects.

HMR stands for Hot Module Replacement. When I save a file, Vite swaps just that module into the running page without a full reload - the browser keeps its state. The analogy I like is changing a wheel on a car while it is still moving. Without HMR, every edit would mean a full page reload, which resets the app and is painfully slow. One important detail: HMR is a development-only feature - in production, Vite runs a build that bundles and minifies everything into static files.

IF THEY PROBE FURTHER

Probe: Vite vs Webpack?

> Webpack is the older tool with heavy configuration; Vite is faster in development because it serves on demand and needs almost no config. Vite is the modern default.

Probe: Does Vite type-check TypeScript?

> No - it only transpiles. That is why my build script runs tsc -b first, then vite build.

Q7 - What AI algorithms did you use

Approx. answer time: 50-60 sec

WHY THEY ASK

Core technical question - expect deep follow-ups on each algorithm.

YOUR SCRIPT

The live game runs Expectiminimax - minimax adapted for games with chance, like dice. For each of my tokens that can move, I compute a heuristic score. Then I look ahead: I simulate what the opponent might do for each of the six possible dice values, take the worst reply for me - with a 70 percent rationality bias, so the simulated opponent plays smart rather than randomly - and average those into an expected value per token. The token with the highest expected value is the best move.

I also implemented two more engines in the codebase. Monte Carlo Tree Search, or MCTS, which plays out hundreds of random games - mine runs 500 iterations - and picks the move that wins most often. And a Hybrid engine that runs both and prefers MCTS when they disagree. In addition, the win probabilities in the UI come from fast Monte Carlo simulation: 100 random playouts of up to 50 turns each. Finally, the heuristic itself has three personalities - aggressive, defensive, and balanced - that retune the weights.

IF THEY PROBE FURTHER

Probe: Why does the live game use Expectiminimax instead of the hybrid?

> It is fast, deterministic, and its reasoning is easy to display in the UI. The hybrid runs two engines, which doubles compute for a small quality gain - and MCTS needs many iterations to become stable.

Probe: What is UCT?

> The formula MCTS uses to balance exploring new moves against exploiting known-good ones - win rate plus an exploration term. My exploration constant is 1.41.

Q8 - Are all the players playing independently

Approx. answer time: 40-50 sec

WHY THEY ASK

Checks whether you understand shared state in the game - a subtle point that impresses.

YOUR SCRIPT

No - and that is the whole point of Ludo. All sixteen tokens live on one shared board inside a single game-state object, and players interact in three main ways. First, captures: if I land on a cell you occupy, your token gets sent back to base. Second, blockades: two of my tokens on the same cell form a wall that blocks you from landing. Third, risk: my AI scans all opponents to check whether any enemy token sits one to six tiles behind mine - that makes my token unsafe.

The AI is even opponent-aware inside the search: to score a move, it literally simulates the opponent's possible replies on the shared board. Each AI player makes its own independent decisions - they do not collude - but every decision reacts to the same shared board.

IF THEY PROBE FURTHER

Probe: Do the AI players team up against the human?

No. Each AI evaluates only its own best move on the shared state - there is no coalition logic anywhere.

Probe: What happens if two tokens share a cell?

If they are mine, it is a blockade - safe from capture and blocks others. If an opponent is already there, I capture it.

Q9 - What did you learn from this project

Approx. answer time: 40-50 sec

WHY THEY ASK

Self-awareness and learning ability - the most important traits for an intern.

YOUR SCRIPT

A huge amount, and I can structure it into three buckets. First, systems thinking: I learned how a full-stack app fits together - that game rules belong on the backend as the single source of truth, and the frontend should only render state and send user actions. Second, API design: defining endpoints like POST /roll and POST /move, agreeing on response shapes in an API contract, and mirroring them as TypeScript types.

Third, AI fundamentals: what a heuristic is, what expected value means, how search trees work, and how randomness changes strategy. I also sharpened practical skills - React state management, TypeScript, CORS, debugging two servers, and writing tests. And honestly, the biggest lesson was that a simple-looking game gets deep very fast when you try to build an AI for it - and that depth is what made the project worth doing.

IF THEY PROBE FURTHER

Probe: Biggest challenge?

Keeping the frontend board in sync with the backend. I solved it by making the backend return the complete updated state on every action, so the frontend never has to guess.

Probe: What would you improve?

Write tests earlier, and design the API contract on paper before writing code - both save time later.

Q10 - Before vs after the project - what learning will you take forward

Approx. answer time: 45-55 sec

WHY THEY ASK

They want a growth story - where you started, where you are now, what sticks.

YOUR SCRIPT

Before this project, I could write small scripts, but I had no real idea how a software system works - what a server actually does, what an API is, or how the pieces of an application talk to each other. After finishing it, I can explain the whole architecture end to end: React frontend, FastAPI backend, REST endpoints, game state, AI evaluation, and what deployment would take.

What I will take forward is the process more than the code. I learned to think before coding - define the rules and the data contract first. I learned to keep the source of truth in one place. I learned to test as I go. And I learned that when you are stuck, breaking the problem into small pieces always works. I will also keep building small AI projects, because the search and heuristic side genuinely fascinated me.

IF THEY PROBE FURTHER

Probe: What would you do differently?

> I would start with a clear API contract on paper, and I would write tests alongside every rule instead of at the end.

Probe: Any specific failure you remember?

> Early on, the frontend and backend drifted out of sync after captures, and the board showed a token in the wrong place. The fix - full state in every response - taught me more about API design than any tutorial.

Q11 - Explain the basic file structure and their roles in simple terms

Approx. answer time: 50-60 sec

WHY THEY ASK

Do you actually know your own project? BNY interviewers drill into files and ask for code snippets.

YOUR SCRIPT

The project has two main folders: backend and frontend. The backend is Python. main.py is the server - it defines the API endpoints like POST /roll and POST /move; think of it as the receptionist that receives requests and routes them. ludo_gcs.py is the heart - the LudoGCS class holds the board state and every game rule, like the referee who knows every rule in the book. ludo_ai.py contains the AI engines - the strategist that evaluates moves. tests.py holds the safety checks, and api_contract.json documents the exact shape of every API response.

The frontend is React with TypeScript. App.tsx is the main app - it owns the game loop and all the calls to the backend. The components folder has Board.tsx, which draws the 15 by 15 board as SVG; Dice.tsx, the animated dice; and StrategyPane.tsx with WinGauge.tsx, which display the AI analytics. schema.ts stores shared types and constants like player colors and speed settings. The rest - vite.config.ts, index.html, package.json - is the standard frontend scaffolding.

IF THEY PROBE FURTHER

Probe: Why is the game logic not in the frontend?

> Single source of truth: rules in one place means no drift between players, harder to cheat, and easy to test in isolation.

Probe: Which file would you change to add a new rule?

> ludo_gcs.py, inside LudoGCS - rules and state all live there, so adding, say, a rule that a six lets you spawn a second token would be one small method.

Q12 - Is this project deployed

Approx. answer time: 30-40 sec

WHY THEY ASK

Honesty check plus practical awareness. Never pretend it is deployed - this answer is designed to impress with self-awareness.

YOUR SCRIPT

Not yet - right now it runs locally, and I can be direct about that. The backend runs on localhost port 8000 and the frontend on port 5173, and the frontend currently has the local backend URL hardcoded. I deliberately spent my effort on the game and the AI rather than deployment.

But I know exactly what deployment would take: host the backend on a cloud platform like Render or Railway, run a production build of the frontend and host the static files on Netlify or Vercel, point the frontend at the deployed backend through an environment variable instead of a hardcoded URL, and widen the CORS whitelist to the new domain. Those are all steps I understand even though I have not executed them yet.

IF THEY PROBE FURTHER

Probe: Why did you not deploy it?

> It was a learning project - my priorities were features and correctness. Deployment is on my roadmap, and I have the plan ready.

Probe: Any blocker you foresee?

> The main risk is the CORS and environment-variable setup, not the hosting itself - but that is exactly the kind of thing I would learn by doing.

Q13 - Do you think with your current skillset you will be able to deploy it

Approx. answer time: 35-45 sec

WHY THEY ASK

Confidence plus honesty balance - they probe for self-awareness and realistic self-assessment.

YOUR SCRIPT

Yes, I believe so - with a little learning, and I can say honestly what I know and what I would pick up. I understand the pieces: building the frontend for production with vite build, pointing the app at a remote API via environment variables, CORS configuration, and running the backend with uvicorn.

What I have not done yet is operate a cloud platform hands-on - creating a project, deploying a service, and managing HTTPS. But that part is a matter of following platform documentation, and I already understand what needs to happen, which is the harder half. If I were asked to deploy it tomorrow, my first steps would be: deploy the backend to Render, verify the API responds from the internet with curl, then host the frontend and connect them, then handle the CORS and HTTPS details.

IF THEY PROBE FURTHER

Probe: What is HTTPS and why does it matter?

> It is the encrypted version of HTTP - it scrambles traffic so nobody can read or tamper with it. Every production site needs it, and platforms like Render and Vercel provide it automatically.

Q14 - Why do you need a database, and why not do it on the client side

Approx. answer time: 40-50 sec

WHY THEY ASK

Data and architecture basics - they want to hear the trade-off reasoning.

YOUR SCRIPT

My game currently has no database - the game state lives in the server's memory, which is fine for a single local demo, and honestly it kept the project simpler. But a database becomes necessary when you want user accounts, saved match history, leaderboards, or letting players resume a game later. And for a real multiplayer product, every game room and its state would need to be stored so players can reconnect.

And why not client-side storage like localStorage? Three reasons. Trust: if the state lives in the browser, a user can open dev tools and cheat - the server must own the state. Sharing: localStorage belongs to one browser on one device - the other player can never see it. Consistency: with state split across clients, the two boards would drift apart. So a server-side database is the only place that is trusted by everyone and shared by everyone.

IF THEY PROBE FURTHER

Probe: Which database would you choose?

> SQLite for the simple local version; for a real multiplayer product, PostgreSQL - it handles concurrent connections and is the standard choice.

Probe: What is localStorage good for then?

> Small non-sensitive preferences, like remembering which speed mode the player last chose. Never game state.

Q15 - What is server-driven and what is client-driven

Approx. answer time: 40-50 sec

WHY THEY ASK

Architecture philosophy - can you explain the big picture in simple words?

YOUR SCRIPT

It is about who makes the decisions. Server-driven means the server owns the logic and the client mostly displays what it is told - like a restaurant where the kitchen decides everything and the waiter only carries dishes. Client-driven means the client does the heavy lifting and the server just stores or serves data - like cooking at home.

My project is strongly server-driven. All game rules, dice rolls, turn order, and AI evaluation happen on the backend. The frontend is a thin client: it renders whatever state the server returns and sends back the user's actions. That was a deliberate choice for three reasons: one source of truth, no cheating, and the ability to redesign the UI without touching game logic. Client-driven makes sense when you want offline apps or minimal server cost - but you give up control and trust.

IF THEY PROBE FURTHER

Probe: What are the downsides of server-driven?

> Every action needs a network round trip, offline play is impossible, and the server does all the work - for a small game those are fine trade-offs.

Probe: Is your frontend client-driven in any way?

> Partially - the frontend drives the game loop, deciding when to trigger AI turns and pacing animations. But every decision about the game itself is the server's.

Q16 - What are REST APIs, FastAPI, and Flask

Approx. answer time: 35-45 sec

WHY THEY ASK

Fundamentals - intern-level, always asked in some form.

YOUR SCRIPT

A REST API is a standard way for two programs to talk over HTTP, using URLs and methods like GET and POST - think of it as a restaurant menu. The client says 'give me the game status' - that is a GET - or 'make this move' - that is a POST - and the server answers in JSON, a simple text format both sides understand.

FastAPI and Flask are both Python frameworks for building REST APIs - different kitchen setups. Flask is the older, minimal framework: tiny and flexible, but you handle validation, documentation, and structure yourself. FastAPI is modern: it automatically validates requests using Pydantic, generates interactive docs for free, supports async for speed, and uses Python type hints end to end. My project uses FastAPI.

IF THEY PROBE FURTHER

Probe: What are the common HTTP status codes?

> 200 OK, 201 Created, 400 Bad Request - which my API uses for 'not your turn' - 401 and 403 for authentication problems, 404 Not Found, and 500 for server errors.

Probe: What is JSON?

> A text format for structured data - like player: P1, token_id: 2 - that both frontend and backend can read without custom parsing.

Q17 - Why did you use FastAPI over Flask

Approx. answer time: 35-45 sec

WHY THEY ASK

Justifying a real choice - they listen for reasoning, not buzzwords.

YOUR SCRIPT

Three reasons. First, automatic documentation - FastAPI generates interactive API docs at the /docs endpoint for free. I used them constantly while building the frontend, because I could test every endpoint in the browser without writing a client. Second, validation - I declare request models with Pydantic, so a wrong payload is rejected automatically with a clear error, instead of manual if-checks like in Flask. Third, modern features - native async support and better performance out of the box.

Flask is a fine tool for tiny microservices, but for an API that handles typed game moves and analytics, FastAPI saved me real time and real bugs.

IF THEY PROBE FURTHER

Probe: Have you actually used Flask?

> I have studied it and its concepts; for this project I chose FastAPI for the reasons above - documentation, validation, and async.

Probe: Any downside to FastAPI?

> It is a bit heavier conceptually - Pydantic models and type hints - and for a hello-world service Flask is simpler. The complexity pays off as the API grows.

Q18 - What other algorithms did you explore, and why did you not go with them

Approx. answer time: 55-65 sec

WHY THEY ASK

Depth plus honesty. This is where you shine - show exploration and trade-offs, not just one answer.

YOUR SCRIPT

I explored three alternatives, and I can be honest about what I kept and what I set aside. First, plain minimax with alpha-beta pruning - the chess approach. I did not use it as the main engine because Ludo has dice: with chance in the game you need chance nodes anyway, and pruning helps little in a shallow, random tree.

Second, Monte Carlo Tree Search - I actually implemented it with 500 iterations and the UCT formula. It is strong, but it needs many iterations to become stable, and it is statistically noisy - for a quick turn-based game, Expectiminimax was faster and its reasoning is far easier to display. Third, a hybrid that runs both engines and prefers MCTS when they disagree - I implemented that too, but running two engines doubles the compute for a small quality gain, so the live game calls Expectiminimax.

I also considered neural networks, the AlphaGo style - but training needs data and compute that a Ludo heuristic does not justify. My trade-off was transparency and speed over raw strength.

IF THEY PROBE FURTHER

Probe: So MCTS is dead code?

> No - it is implemented and testable in the codebase; the live path just does not call it. The hybrid can be enabled with a few lines if I ever want to compare.

Probe: When would you use MCTS?

> For games with huge branching where a good evaluation function does not exist - like Go - or where random playouts are cheap and fast.

Probe: What is alpha-beta pruning?

> A way to skip branches that cannot affect the result - like the lazy shopper who found a 50-dollar jacket and stops checking shops that only sell more expensive ones. Same answer as full minimax, but far faster.

Q19 - Can you tell me about the API contract

Approx. answer time: 45-55 sec

WHY THEY ASK

API design awareness - BNY loves 'explain your contract' questions.

YOUR SCRIPT

The API contract is the agreement between the frontend and backend about how they talk - every endpoint, what it accepts, and what it returns. I documented it in a file called api_contract.json. There are eight endpoints. GET / returns the initial board state. GET /game-status returns who is winning and the active player. GET /legal-moves/{player}/{roll} returns the legal moves for a player given a roll. POST /roll rolls the dice and returns the roll plus a full move analysis. POST /move executes a move. POST /pass skips a turn when no moves are available. POST /reset starts a new game. And POST /configure sets which players are AI and what strategy each uses.

The most important one is POST /roll: it returns the roll, the full board state, an analysis of every legal move with scores and reasoning, and metadata like the best move and win probabilities. The frontend simply follows this contract - it never invents data.

IF THEY PROBE FURTHER

Probe: Why a separate contract file?

So both sides agree on shapes - I mirror it as TypeScript types in schema.ts, and the backend validates with Pydantic, so mismatches fail early.

Probe: Which endpoint did you change most?

POST /roll - I kept adding analytics fields as the UI evolved, which is exactly why having a written contract helped.

Q20 - Why did you choose this API contract

Approx. answer time: 40-50 sec

WHY THEY ASK

Design reasoning - the 'why' behind the shapes.

YOUR SCRIPT

The contract is shaped by one principle: the backend is the single source of truth and the frontend stays thin. So every endpoint that changes the game returns the complete updated state - the board, the active player, and the game status - which means the frontend never has to compute or guess anything; it just renders whatever it receives. That single decision removed an entire class of sync bugs.

Second, I bundle the analytics with the roll instead of making a separate call - one round trip instead of two, which keeps the UI feeling instant. Third, requests are tiny and typed: player, token_id, roll - validated by Pydantic on the backend and mirrored as TypeScript types on the frontend. And fourth, every mutating endpoint checks server-side whose turn it is, so a client cannot move out of turn or fake a roll.

IF THEY PROBE FURTHER

Probe: Trade-offs of this design?

Responses are larger because they carry the whole state, and the server does more work per call - both are fine at this scale and they buy simplicity.

Probe: Would you change it for multiplayer?

Yes - I would add a game ID to every request so state is per-room, and I would probably add WebSockets for instant updates between players.

Q21 - Webhook, WebSocket, HTTPS connection - what is the difference

Approx. answer time: 40-50 sec

WHY THEY ASK

Networking fundamentals - the classic triad question.

YOUR SCRIPT

Three different ways of communicating, and I will use three analogies. HTTPS is a sealed letter sent by courier: the client sends a request, the server replies, one exchange at a time - and the connection is encrypted, so nobody can read it in transit. In production, my REST API would be served over HTTPS.

A WebSocket is a phone call: one connection that stays open in both directions, so either side can send messages anytime with low latency - perfect for live games, chat, and dashboards. I do not use it in my project because Ludo is turn-based: one request-response per move is enough.

A webhook is a doorbell: instead of the client polling - asking 'did anything happen?' over and over - the client gives the server a URL, and the server rings it when an event occurs, like a payment notification. Webhooks are how systems notify each other without being asked.

IF THEY PROBE FURTHER

Probe: When would you add WebSockets to this game?

For real multiplayer - every player's board should update the instant someone moves, and the server should push instead of waiting to be asked.

Probe: What is polling?

Repeatedly asking the server if something changed. It works but wastes requests - webhooks flip it around.

Q22 - What are the roles and responsibilities of your frontend and backend

Approx. answer time: 40-50 sec

WHY THEY ASK

Architecture clarity - the split must be crisp in your head.

YOUR SCRIPT

The backend is the authority - the referee and the strategist. It owns the game rules: when tokens can move, captures, blockades, the home stretch. It rolls the dice, so nobody can fake a roll. It decides whose turn it is. It runs the AI, computes scores and win probabilities, and reports game status.

The frontend is the presentation and control - the scoreboard, the controller, and the commentator. It draws the board, plays the dice animation, captures clicks, and drives the game loop: when the active player is an AI, the frontend automatically calls roll, then move. It shows the analytics panel that explains the AI's reasoning, and it applies pacing settings like cinematic mode. But it never decides anything about the rules - every decision about the game belongs to the server.

IF THEY PROBE FURTHER

Probe: Who actually picks the AI's move?

The backend computes the best move during POST /roll; the frontend simply executes it. The brain is server-side, the hands are client-side.

Probe: Can a player cheat?

Only by editing what their own screen shows - the server validates every move and every turn, so real cheating is prevented.

Q23 - Why did you decide this split of roles between frontend and backend

Approx. answer time: 40-50 sec

WHY THEY ASK

Justification - can you defend the design with reasoning?

YOUR SCRIPT

Three ideas drove it. Single source of truth: with all rules in one place, there is no drift between players and no cheating - a user can open developer tools and change what their screen shows, but the server still rejects anything invalid. Separation of concerns: the UI only handles showing and collecting input, so I can restyle the entire frontend without touching a single rule - and I can test the rules in isolation in Python. And the right tool for the job: Python is ideal for algorithm-heavy AI code, while React is the best tool for interactive UI.

This split also helped me as a learner - I could test the backend with tests.py before the UI even existed, which made debugging far easier.

IF THEY PROBE FURTHER

Probe: Downsides of the split?

Every action is a network round trip, so the UI waits for the server - fine for a board game, bad for a fast-action game.

Probe: What if the server dies mid-game?

The game state is lost, because there is no database - that is the honest limitation, and a database plus sessions would fix it.

Q24 - Could you change the frontend and backend roles to make it more optimal

Approx. answer time: 45-55 sec

WHY THEY ASK

Flexibility plus trade-off thinking - they want to hear you reason, not defend.

YOUR SCRIPT

Yes, and I have thought through the options and their trade-offs. Option one: move move-validation to the frontend for instant feedback - the UI would feel snappier, but a user could tamper with the state, so the server would have to validate anyway - duplication without security gain.

Option two: move the AI brain to the frontend - fewer API calls and offline play - but the strategy code would be exposed in the browser, hard to update centrally, and Python's strengths would be wasted.

Option three: keep the rules server-side but add WebSockets so the server pushes state changes instead of the client requesting them - that is the upgrade I would actually make for a multiplayer version. So for this project the current split is the right one - authority on the server, experience on the client. I would only change it if the requirements changed.

IF THEY PROBE FURTHER

Probe: Would you ever move logic to the client?

For a competitive game, trust matters most, so no. For a tool or an offline app, absolutely - it depends on the trust model.

Probe: What is the most valuable change you would make first?

Sessions and per-game state - so two players could each have their own game. That is the biggest structural weakness today.

Q25 - What do you mean by low latency communication

Approx. answer time: 30-40 sec

WHY THEY ASK

Systems thinking - do you understand performance beyond 'it works'?

YOUR SCRIPT

Latency is the delay between doing something and seeing the result - like the lag on a video call. Low latency means that delay is small enough that the experience feels instant. In my game, every action is one HTTP round trip: I click a token, the request goes to the server, the server validates and computes, the response comes back, and the board updates. On localhost that is near-zero, so it feels instant.

Latency matters most in real-time games like shooters, where milliseconds decide outcomes. Ludo is turn-based, so even a few hundred milliseconds is fine - in fact I add artificial delays through speed modes like cinematic to make the game feel more dramatic, which is the opposite problem.

IF THEY PROBE FURTHER

Probe: How would you measure it?

Browser developer tools show the timing of every request - I used them while tuning the AI search time.

Probe: What affects latency here?

Network distance, server compute time - mainly the AI search - and in production, HTTPS handshakes and hosting location.

Q26 - How are your frontend and backend synchronized? Are you maintaining state between them?

Approx. answer time: 45-55 sec

WHY THEY ASK

THE architecture question for this project - nail this one.

YOUR SCRIPT

We are not polling, and there is no WebSocket - synchronization happens through the request-response pattern itself. The backend holds the single source of truth: one game-state object. The frontend keeps a mirror copy in React state. Every action - roll, move, pass, reset - is a request to the server, and the server's response always contains the complete updated state: the board, the active player, and the game status. The frontend simply merges that response into its mirror, so the two are always identical - by construction, not by syncing.

The AI turn shows this clearly: the frontend sees the active player is an AI, calls POST /roll, the server computes the best move, and the frontend sends POST /move. Every response updates React state, which re-renders the board. One honest limitation: the state is global on the server, so two browser tabs would fight over the same game - acceptable for a demo, and in a real product I would add game sessions.

IF THEY PROBE FURTHER

Probe: So the frontend never holds truth?

Correct - it holds a mirror. If the backend and frontend ever disagreed, the backend wins, because every response overwrites the mirror.

Probe: Why not just poll the server for state?

Polling would waste requests and still not be real-time. Since every action already returns full state, polling adds nothing.

Part 2 - Bonus technical questions (B1 - B6)

B1 - What is CORS and why did you need it

Approx. answer time: 30-40 sec

WHY THEY ASK

Extremely common follow-up - your project literally configures it.

YOUR SCRIPT

CORS is a browser security rule. A web page served from localhost:5173 is not allowed to read responses from localhost:8000 - a different origin - unless that server explicitly allows it. Without it, the browser silently blocks my API responses.

So in the backend I added FastAPI's CORSMiddleware and whitelisted the frontend origins - localhost:5173 and 127.0.0.1:5173. It is like a security guard at the server's door checking a guest list. The important detail is that the fix belongs on the server, not the frontend - and in production I would whitelist the deployed frontend's domain.

IF THEY PROBE FURTHER

Probe: Why does Postman work but the browser fails?

> Postman is not a browser, so it does not enforce CORS - it is the browser that enforces the rule, not the server.

Probe: What does 'origin' mean?

> The scheme plus host plus port - so localhost:5173 and localhost:8000 are different origins even on the same machine.

B2 - How would you turn this into a real multiplayer game

Approx. answer time: 40-50 sec

WHY THEY ASK

The classic scale question - BNY reports confirm 'how would you scale this 10x?' appears.

YOUR SCRIPT

I would turn the single global game into game rooms. Each room gets a unique ID and its own game state, and each player gets a session that links them to a room. I would add a database to persist rooms, match history, and leaderboards. I would switch the frontend from pull-based fetch to WebSockets, so every player's board updates the instant anyone moves.

I would add matchmaking to pair opponents, turn timers so games do not stall, and reconnection so a dropped player can rejoin. Server-side validation stays, so cheating stays hard. And I would deploy properly: backend on a cloud host, frontend on a static host, with HTTPS. The good news is the core rules and the AI survive unchanged - it is an expansion, not a rewrite.

IF THEY PROBE FURTHER

Probe: What breaks today if two people play?

> They share the one global game state on the server - moves conflict. That is exactly what rooms and sessions fix.

Probe: Where would the rooms live?

> In memory for a small prototype, and in the database once they need to survive restarts.

B3 - Why did you render the board with SVG and not Canvas

Approx. answer time: 35-45 sec

WHY THEY ASK

Rendering choice - shows practical frontend judgment.

YOUR SCRIPT

SVG is vector shapes living in the HTML - every cell and token is an object with properties. Canvas is a drawing surface - you paint pixels, and afterwards there is no structure left. For a board game with a 15 by 15 grid and sixteen tokens, SVG is the better fit: it stays crisp at any size, it is easy to style with CSS, and most importantly each token can have its own click handler and animation - which I used with Framer Motion for the spring-in effects and the pulsing gold ring on the best move.

Debugging is also trivial because you can inspect elements in the browser. Canvas shines for thousands of moving particles or frame-perfect rendering - like games with many sprites - which a board game simply does not need.

IF THEY PROBE FURTHER

Probe: Any downside to SVG?

Performance - thousands of DOM elements get slow. Sixteen tokens are nothing, so SVG is the right call here.

Probe: What does the pulsing gold ring show?

The AI's best move - computed by the backend and rendered by the frontend.

B4 - How did you test your rules and AI

Approx. answer time: 30-40 sec

WHY THEY ASK

Interns rarely test - showing tests is a differentiator.

YOUR SCRIPT

I wrote backend/tests.py - plain assert-based tests with no framework, so they run anywhere with a single command: python tests.py. They cover rule behaviour - things like legal moves, captures, and turn logic - and they include a performance guard: the AI search must complete within 250 milliseconds, so the game stays responsive, since the AI runs on every roll.

The tests caught real regressions while I refactored the engine, which taught me to respect them. If the project grew, I would move to pytest with CI so every push runs them automatically, and I would add frontend tests with Vitest.

IF THEY PROBE FURTHER

Probe: What does CI mean?

Continuous Integration - automated tests run on every change before it is merged, so bugs surface immediately.

Probe: Why no framework?

For this size, plain asserts kept the setup zero-dependency - python tests.py just works. Pytest would be my choice once the suite grows.

B5 - What is the time cost of your AI thinking

Approx. answer time: 30-40 sec

WHY THEY ASK

Complexity awareness - pair this with the cheat-sheet numbers.

YOUR SCRIPT

For each legal token move, the engine computes a heuristic score, then simulates the opponent's reply across all six dice values, each with four tokens - so it is roughly moves times 6 times 4 evaluations per roll. That is a small, bounded number, because the game state is tiny - sixteen tokens.

In practice it completes in a few milliseconds, and my test suite enforces a 250 millisecond upper bound so it can never drag. If I ever wanted deeper search, I would add caching or pruning - but at this scale it is unnecessary, and the responsiveness is better for the demo.

IF THEY PROBE FURTHER

Probe: What is the branching factor here?

At most four tokens times six dice per player - tiny compared to chess at about 35, or Go at about 250.

Probe: How does depth change the cost?

Each extra ply multiplies the work by the branching factor - which is exactly why deep search is expensive in games like chess.

B6 - What happens if two people open your app at once

Approx. answer time: 30-40 sec

WHY THEY ASK

Honest-limitation question - shows self-awareness, a BNY integrity theme.

YOUR SCRIPT

They would share one game. The backend keeps a single global game state with no sessions or rooms, so two tabs are effectively playing the same game and could even conflict over whose turn it is - one move request would be rejected as 'not your turn'.

That is a known limitation I accepted for a local demo, and the fix is the multiplayer design I described earlier: per-game state keyed by a room ID, sessions to assign players, and a database if games must survive restarts. I like this question because it lets me show that I understand why production systems need sessions - it is exactly the kind of scale question a bank would care about.

IF THEY PROBE FURTHER

Probe: Is that a bug?

For a demo, it is a design limitation. For a product, it would be a bug - and the fix is rooms and sessions.

Probe: How would you demo it to a friend?

Each person runs the frontend and backend on their own machine - since the game is local, one machine, one game.

Part 3 - HR and behavioural questions (H1 - H5)

These come straight from real BNY internship reports - the first three are near-guaranteed. Answer every one with a specific example from your own experience; never invent stories.

H1 - Why BNY Mellon

Approx. answer time: 40-50 sec

WHY THEY ASK

THE most-reported BNY question - a research-backed answer is mandatory.

YOUR SCRIPT

Three reasons. First, what BNY is: it is one of the world's largest custodian banks - holding trillions of dollars in assets - and it has been around for more than two centuries. That scale means its technology handles problems most companies never face: reliability, security, and trust at massive volume, which genuinely excites me.

Second, what the role offers me: an internship where I can learn from experienced engineers while working on systems that matter, and grow into a full-stack developer with real production experience. Third, what I offer BNY: I am a fast, hands-on learner - my Ludo project proves I can take an idea from a blank page to a working full-stack application with AI - and I bring curiosity and energy into a team that values both.

IF THEY PROBE FURTHER

Probe: What do you know about BNY technology?

> It runs treasury services, asset servicing, and payments at enormous scale - and my project taught me how much thought goes into correctness and state management, which is the same discipline at a far bigger level.

Probe: Why not a startup?

> I want to learn fundamentals from senior engineers on systems where correctness is critical - that is where I will grow the fastest as an intern.

H2 - What are your strengths and weaknesses

Approx. answer time: 40-50 sec

WHY THEY ASK

Self-awareness check - the weakness must be real, non-core, and paired with a fix.

YOUR SCRIPT

Two strengths. I am a hands-on, end-to-end learner - I do not feel I understand something until I have built it, and my project is the evidence: frontend, backend, and AI in one application. And I am systematic when stuck - I break problems into small pieces and test each one, which is why my game's rules have a test file and a performance guard.

My honest weakness: I sometimes go too deep into polishing details - for example, I spent extra time tuning the AI personalities when a simpler version would have been fine. I am working on it by timeboxing tasks and asking 'what is the minimum version that works?' before starting - so I can trade polish for progress when it matters.

IF THEY PROBE FURTHER

Probe: How do you learn a new technology fast?

> I read the official docs, build the smallest possible working example, then extend it - docs plus hands-on beats tutorials for retention.

Probe: Do you prefer working alone or in a team?

Both - I built this project alone, but I like team settings because reviewing each other's code makes everything better, and I would love that at BNY.

H3 - Where do you see yourself in five years

Approx. answer time: 35-45 sec

WHY THEY ASK

Ambition plus realism - move up one or two levels, never claim their job.

YOUR SCRIPT

In five years I want to be a confident full-stack engineer who can own a feature end to end - from design to deployment - and who is trusted with increasingly important systems. In the near term, my goal is to complete this internship having shipped real work and learned production engineering - code review, testing, deployment, and working in a team.

From there I want to grow within the company, deepening my skills in backend and AI, and eventually take on more responsibility - mentoring juniors the way I hope to be mentored now. I am not someone who plans to jump around: I would rather grow deep inside one organisation that invests in me, and BNY's scale and stability make that path real.

IF THEY PROBE FURTHER

Probe: What if the internship has no full-time offer?

I would take the experience - the fundamentals I learn here are portable - and keep building. But my first choice is to earn that offer.

Probe: Management or individual contributor?

Individual contributor first - I want technical depth before any leadership role.

H4 - Tell me about a time you handled a disagreement or a conflict of priorities

Approx. answer time: 40-50 sec

WHY THEY ASK

BNY explicitly asks about team conflict - prepare a real, honest story using STAR.

YOUR SCRIPT

Let me share a project example. Situation: while building the Ludo game, I wanted to add a visual analytics panel, but I was also mid-way through wiring the AI to the API - a real conflict of priorities inside my own project. Task: decide which came first, since both could not be done well in the same session. Action: I paused and wrote out the dependencies - the AI had to work before the panel could show anything meaningful - so I finished the AI integration first and made the panel a stretch goal.

Result: the game worked end to end sooner, and the panel shipped right after, using real data instead of fake numbers. The lesson I took forward: when priorities collide, order them by dependency - build the thing everything else needs first.

IF THEY PROBE FURTHER

Probe: What about a disagreement with a real person?

I do not have a corporate example yet - as an intern I will likely face that for the first time. My instinct is to understand their constraints first, then find the common goal, because most disagreements in code are about trade-offs, not ego.

Probe: How do you handle feedback?

I treat it as free information - the fastest way I know to improve is someone else's honest review, so I ask for it early and often.

H5 - How have you dealt with ethics and integrity

Approx. answer time: 40-50 sec

WHY THEY ASK

BNY's most distinctive question - banks screen for integrity explicitly. Answer with a concrete, true story.

YOUR SCRIPT

This connects directly to my project. I built most of my code with the help of AI tools, and I decided early to be completely honest about that in this interview - because integrity is something you cannot fake, and a bank that handles trillions of dollars needs people who value trust.

Concretely: I made sure I understood every line of code I shipped - if I cannot explain a piece of code, it is not mine, and I do not let it in. I also kept the project truthful in another way - the documentation claims match the code. When I found that my notes described a depth of search the code did not actually implement, I fixed the documentation rather than quietly leaving a mismatch. To me that is integrity in practice: the record must match reality, even when nobody would notice.

IF THEY PROBE FURTHER

Probe: Would you use AI in this job?

Yes - as a tool to move faster - but always with review, verification, and full transparency with my team. The code I ship is code I understand.

Probe: What does integrity mean to you?

Doing the right thing when nobody is watching - and my documentation fix is the smallest possible example of that habit.

Part 4 - Cheat sheet (memorize these numbers)

Board numbers

Topic	Number	Meaning
Base	-1	Token in base - needs a 6 to enter the board
Main track	0 to 51	The 52-cell loop around the board
Home stretch	52 to 56	Token's own column - always safe
Home (goal)	57	Token finished; distance to goal = 57 - position
Start offsets	P1=0, P2=41, P3=27, P4=13	Where each player enters the track
Star safe tiles	1, 8, 14, 21, 28, 35, 42, 49	8 safe cells on the track
Rules	6 to spawn; 6/capture/home = bonus roll; 3 sixes = turn void; exact roll to finish; 2 tokens on a cell = blockade	

The score formula (balanced weights)

Score = (position x 2) + 30 (in home stretch) + 200 (at home) + 25 (safe tile) - 35 (enemy 1 to 6 tiles behind) + 40 per blockade + 80 per capture. Final move score = expected value across all six dice outcomes.

Personality weights (the three play styles)

Weight	Balanced	Aggressive	Defensive
Capture bonus	80	150	50
Risk penalty	35	15	50
Progress	2.0	2.5	2.0
Safety	25	10	40
Blockade	40	30	60
Home bonus	200	200	250

AI engine numbers

Topic	Number / fact
Live engine	Expectiminimax - 1 move ahead + opponent's reply on all 6 dice
Rationality bias	70 percent (opponent plays smart, not random)
MCTS	500 iterations, UCT exploration constant 1.41, 30-turn playouts
Win probability	100 random playouts x up to 50 turns each
Performance guard	AI search must finish under 250 ms (enforced by a test)
Branching	At most 4 tokens x 6 dice per player (chess ~35, Go ~250)

API endpoints (8 total)

Method	Path	Purpose
GET	/	Initial board state
GET	/game-status	Status and winner
GET	/legal-moves/{player}/{roll}	Legal moves for a roll
POST	/roll	Roll dice + move analytics + best move
POST	/move	Execute a move
POST	/pass	Skip turn
POST	/reset	New game
POST	/configure	Set AI players and strategies

Stack and project facts

Topic	Fact
Backend	Python + FastAPI + Uvicorn, port 8000
Frontend	React 19 + TypeScript + Vite 8 + Framer Motion, port 5173
Communication	Plain HTTP + JSON via fetch - no WebSockets
State	In-memory on the server - no database
Deployment	None - runs locally on localhost only
Tests	backend/tests.py - plain asserts, run with: python tests.py
AI personalities	Aggressive / defensive / balanced
Analogy bank	Restaurant kitchen = server-driven; sealed letter = HTTPS; phone call = WebSocket; doorbell = webhook;

Part 5 - One-page architecture summary

The big picture

Two applications talk over HTTP. The backend (Python, FastAPI, port 8000) is the authority: it owns every game rule and the AI brain. The frontend (React + TypeScript, Vite, port 5173) is thin: it renders the board, captures clicks, and drives the game loop. The backend holds one global game-state object (LudoGCS); the frontend keeps a mirror in React state that every API response overwrites - so they are always identical by construction.

The data flow

Player clicks Roll - POST /roll - the server rolls the dice, runs Expectiminimax analytics and 100 Monte-Carlo win simulations - and returns the roll, the full board state, per-move scores with reasoning, the best move, and win probabilities. The UI highlights the best move in gold. The player clicks a token - POST /move - the server validates the move against every rule, resolves captures and blockades, decides bonus-roll or turn advance, and returns the complete updated state. The frontend merges it, React re-renders, and if the next player is an AI, the frontend repeats the same flow automatically.

Why the split works

- Authority on the server: rules, dice, turn order, and AI evaluation all live server-side - one source of truth, no cheating, easy to test.
- Experience on the client: animations, pacing (speed modes), and the analytics panel live client-side - the UI can be redesigned without touching game logic.
- The AI brain is server-side; the frontend simply executes the best move the backend computes.

Honest limitations (say these confidently)

- One global game - no sessions or rooms, so two tabs share a game (fix: per-game state keyed by room ID).
- State is in memory - a server restart wipes the game (fix: a database).
- Runs only on localhost - not deployed (deployment plan in Q12).
- MCTS and the hybrid engine are implemented and tested, but the live path calls Expectiminimax.
- Early documentation over-claimed search depth and pruning - the docs were corrected to match the code.

File map

File	Role in plain words
backend/main.py	The server - API endpoints, CORS, turn progression
backend/ludo_gcs.py	The heart - board state, all rules, heuristic scoring, win simulations
backend/ludo_ai.py	The brain - Expectiminimax, MCTS, hybrid engines
backend/tests.py	Safety checks - rules + 250 ms AI speed guard
backend/api_contract.json	The written agreement on every response shape
frontend/src/App.tsx	Main app - game loop, all backend calls, UI wiring
frontend/src/components/Board.tsx	Draws the 15x15 SVG board and tokens
frontend/src/components/Dice.tsx	The 3D animated dice
frontend/src/components/StrategyPane.tsx, WinGauge.tsx	The AI analytics dashboard
frontend/src/types/schema.ts	Shared TypeScript types and constants